Experiment 7

Establish a local Blockchain using Hardhat and deploy a Smart Contract

Aim

To establish a local Ethereum blockchain using Hardhat and deploy a set of Solidity smart contracts modeling a 4-stage supply chain (Farmer → Distributor → Retailer → Customer), then interact with the deployed contracts to trace one batch through the full chain.

Theory

Hardhat is a local Ethereum development environment that compiles Solidity contracts, runs a full in-process (or standalone) Ethereum node for testing, and lets contracts be deployed and interacted with entirely offline using deterministic, pre-funded test accounts — the same role Ganache plays, but as a Node.js-native toolchain with first-class scripting support via `ethers.js`. `npx hardhat node` starts a standalone JSON-RPC blockchain on `http://127.0.0.1:8545` seeded with 20 accounts of 10,000 test ETH each; `npx hardhat run <script> --network localhost` then connects to that running node exactly as Remix-with-Metamask or a real dApp frontend would, deploying contracts and sending transactions that are mined into real, numbered blocks with real gas costs — all without needing a wallet extension, a faucet, or a public testnet.

The task's contract is a 4-stage supply-chain traceability system, four contracts chained by constructor-injected addresses:

- **`FarmerContract`** — `addBatch(id, cropName, quantity)` records a new batch on-chain with the farmer's address and a timestamp; `getBatch` reads it back.
- **`DistributorContract`** — holds a reference to `FarmerContract` and `receiveBatch(id)` calls into it via the `IFarmerContract` interface to pull the batch's data and record that this distributor received it.

- **`RetailerContract`** — same pattern one stage further down, reading from `DistributorContract` via `IDistributorContract`.
- **`CustomerContract`** — the final stage, reading from `RetailerContract` via `IRetailerContract` and recording the purchase.

Each stage's `require` guards (batch must exist upstream, must not already have been received/purchased at this stage) enforce that the batch can only move forward through the chain once, and each contract only trusts the *interface* of the contract before it — it never touches the earlier contracts' internal storage — which is the standard pattern for composing several independently-deployed contracts into one traceability pipeline.

Tools used: Node.js 24, Hardhat 2.22 with `@nomicfoundation/hardhat-toolbox` (bundles `ethers.js` v6), Solidity 0.8.24. All commands were run directly in a terminal against a standalone `npx hardhat node` — in place of Ganache + Metamask + Remix IDE, since this was done without a browser wallet.

Output

1. Compile the 4 contracts

```
$ npx hardhat compile
Downloading compiler 0.8.24
Compiled 4 Solidity files successfully (evm target: paris).
```

2. Start a local Hardhat blockchain node

```
$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========
WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
... (20 accounts total, 10000 ETH each)
```

The local blockchain, running as a persistent standalone node rather than the in-memory test network, so a separate deploy script can connect to it the same way a real dApp would.

3. Deploy all 4 contracts and run the full Farmer → Distributor → Retailer → Customer trace

```
$ npx hardhat run scripts/deploy_and_interact.js --network localhost

=== Accounts (Hardhat local network) ===
Farmer     : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Distributor: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
Retailer   : 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
Customer   : 0x90F79bf6EB2c4f870365E785982E1f101E93b906

=== Deploying contracts ===
FarmerContract deployed at     : 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
DistributorContract deployed at: 0x948B3c65b89DF0B4894ABE91E6D02FE579834F8F
RetailerContract deployed at   : 0x8438Ad1C834623CfF278AB6829a248E37C2D7E3f
CustomerContract deployed at   : 0xCba6b9A951749B8735C603e7fFC5151849248772

=== Stage 1: Farmer creates batch #1 (Alphonso Mango, 500kg) ===
addBatch tx hash: 0x8f729e213957ba12bfbab88849e8976173dc22d8d96d56c0f4d8d1445be904aa
FarmerContract.getBatch -> {
  batchID: '1', cropName: 'Alphonso Mango', quantity: '500',
  timestamp: '1788935030', farmer: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266'
}

=== Stage 2: Distributor receives the batch ===
receiveBatch tx hash: 0x60478bad6d49c85e2a1e47fb0e24b1e7851e3f17069c914815e7ab48e3dbea2e
DistributorContract.getReceivedBatch -> { ..., distributor: '0x70997970C51812dc3A010C7d01b50e0d17dc79C8' }

=== Stage 3: Retailer receives the batch from the distributor ===
receiveFromDistributor tx hash: 0x517ff8c1070bc94ba790b9577dc7ba4082c0f7cd96712aa429da10f00441153c
RetailerContract.getRetailBatch -> { ..., retailer: '0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC' }

=== Stage 4: Customer purchases the batch from the retailer ===
purchaseFromRetailer tx hash: 0xb17383ad642690de2091f34182b6d9455fb34aea4582a6506727eb6b88886221
CustomerContract.getPurchasedBatch -> { ..., customer: '0x90F79bf6EB2c4f870365E785982E1f101E93b906' }

=== Negative test: re-purchasing the same batch must revert ===
Reverted as expected: VM Exception while processing transaction: reverted with reason string 'Batch already
purchased!'

=== Full supply chain trace for batch 1 complete ===
```

Full console output of `scripts/deploy_and_interact.js`, saved as `deploy_output.log` in this folder: all 4 contracts deployed, batch #1 walked through all 4 supply-chain stages with each stage's on-chain read confirming the previous stage's data, and a negative test confirming a batch cannot be purchased twice.

4. Corresponding entries from the Hardhat node's own log (`hardhat_node.log`, this folder)

```
eth_sendTransaction
  Contract deployment: FarmerContract
  Contract address:    0x9fe46736679d2d9a65f0992f2272de9f3c7fa6e0
  Gas used:             749765 of 16777216
  Block #10:            0x818c6d0fb2c0b70119afcca3475ce480f5d38c308940d3241dfe6f220e41d988
...
```

```
eth_sendTransaction
  Contract call:       CustomerContract#purchaseFromRetailer
  Gas used:            48994 of 16777216
  Block #18:           0x6cd030bb2267b9a4d640905c20e318370e00fb251901faaff1603ad683c845ea
  Error: reverted with reason string 'Batch already purchased!'
```

The node itself independently confirms each deployment and call as a mined block with real gas accounting, and shows the final repeat-purchase attempt reverting on-chain rather than merely erroring client-side.

Conclusion

This experiment established a local Ethereum blockchain with Hardhat and deployed a 4-contract supply-chain traceability system to it, in place of Ganache + Metamask + Remix IDE (no browser wallet was used in this run). A single batch was walked through all four stages — Farmer, Distributor, Retailer, Customer — with each stage's contract independently verifying the previous stage's on-chain data through its interface before recording its own step, and each of the 4 deployments plus 4 stage-transitions was mined as a distinct, gas-accounted block on the local node (confirmed independently in both the deploy script's own output and the node's server-side log). The final negative test — attempting to purchase the same batch a second time — reverted with `"Batch already purchased!"`, confirming the `require` guards correctly enforce that a batch can only progress through the chain once. This demonstrates the same guarantees a Ganache/Metamask/Remix workflow would (a persistent local chain, real transactions, real gas costs, and enforceable contract state) while remaining fully scriptable and reproducible from the command line.